

Reviewer Pack — Minimal Rank of Tropical Curves Bounds Communication Complex...

Entry 3b39b62d7bd5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 21:51:36 UTC

Minimal Rank of Tropical Curves Bounds Communication Complexity for Read-Twice BP

Entry ID: 3b39b62d7bd5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 21:51:36 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Tropical Curves) **Field B** (complexity object): Communication Complexity (Read-twice BP)

Statement:

For every n , the communication complexity of the read-twice branching program for XOR Boolean functions is lower bounded by a function proportional to the minimal rank of a certain tropical curve associated with the function.

Rationale (proposer's reasoning):

Tropical geometry has been successfully applied to analyze arithmetic circuits and their communication complexity. Tropical curves provide a way to encode combinatorial structures, which could potentially expose hidden complexities in read-twice BP that are not evident through traditional circuit analysis.

Taxonomy category: Read_twice_BP (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b501bd2d76c7d8fa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between communication complexity and minimal rank of tropical curves is ≥ 0.8 for all $n \leq 40$, and the mean difference between them is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "minimal rank" AND "communication complexity"
- "read-twice BP" AND "XOR Boolean functions" AND "minimal rank tropical curves" - "tropical curve" AND "communication complexity lower bound" AND "read-twice branching program"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_xor_function(n):
        return [random.randint(0, 1) for _ in range(n)]

    def generate_tropical_curve(n, f):
        curve = []
        for i in range(2**n):
            binary_rep = format(i, '0{}b'.format(n))
            input_vector = list(map(int, binary_rep))
            curve.append(f(input_vector))
        return curve

    def communication_complexity(curve):
        n = len(curve)
        total_bits = sum(curve)
        return total_bits / n

    def minimal_rank_tropical_curve(curve):
        n = len(curve)
        matrix = []
        for i in range(n):
            row = [curve[j] ^ curve[i] for j in range(n)]
            matrix.append(row)

        rank = 0
        for i in range(n):
            if all(matrix[j][i] == 0 for j in range(i, n)):
```

```

        continue
    pivot_row = i
    for j in range(i+1, n):
        if abs(matrix[j][i]) > abs(matrix[pivot_row][i]):
            pivot_row = j
    matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
    rank += 1
    for j in range(n):
        if i != j:
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    return rank

n_values = [5, 10, 15, 20, 30, 40]
communication_complexities = []
ranks = []

for n in n_values:
    f = generate_xor_function(n)
    curve = generate_tropical_curve(n, f)
    comm_comp = communication_complexity(curve)
    rank_val = minimal_rank_tropical_curve(curve)

    communication_complexities.append(comm_comp)
    ranks.append(rank_val)

mean_comm_comp = sum(communication_complexities) / len(communication_complexities)
mean_rank = sum(ranks) / len(ranks)
diff_mean = abs(mean_comm_comp - mean_rank)

correlation_bound = 0.8
acceptance_threshold = 3

conjecture_holds = (diff_mean <= acceptance_threshold and
                    all(corr >= correlation_bound for corr in [comm_comp / rank_val for comm_comp, rank_val in zip(communication_complexities, ranks)]))

counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Communication Complexity vs Minimal Rank",
    "metric_value": mean_comm_comp,
    "instances_tested": len(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}}, **result}}")
        results.append(result)

```

```

mean_comm_comp = sum(r["metric_value"] for r in results) / len(results)
std_comm_comp = math.sqrt(sum((r["metric_value"] - mean_comm_comp)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_comm_comp} std={std_comm_comp} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_comm_comp} std={std_comm_comp} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, r in zip(seeds, results) if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_68d78220.py", line 100, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_68d78220.py", line 67, in run_trial
    curve = generate_tropical_curve(n, f)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_68d78220.py", line 29, in generate_tropical_curve
    curve.append(f(input_vector))
                  ^^^^^^^^^^^^^
TypeError: 'list' object is not callable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the correlation coefficient and mean difference could not be calculated to verify the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again to obtain the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12406
2	propose	ollama_remote	glm4:latest	0	0	15413
3	propose	ollama_remote	glm4:latest	0	0	9054
4	preregistration	ollama_remote	glm4:latest	0	0	5358
5	novelty	ollama_remote	glm4:latest	0	0	4854
6	novelty	ollama_remote	glm4:latest	0	0	5974
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15919
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16698
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11659
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12383
11	judge	ollama_remote	glm4:latest	0	0	16508

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126226 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/3b39b62d7bd5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3b39b62d7bd5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3b39b62d7bd5.tar.gz` (if generated)